Trainable Input Memories and Skill Binding in a Frozen Language Model: An Empirical Study of Vocabulary Transfer and Isolated Retention

Shenghe Li
Independent Researcher

October 10, 2026

Abstract

Can a small, separately stored input vector bind a frozen language model to a reusable command? We report a retrospective, auditable case study using a previously adapted SmolLM2-1.7B-Instruct reader. A single trainable 2,048-dimensional input vector (8 KiB in FP32) was optimized for a four-word permutation command. On 192 queries containing words excluded from that command's training vocabulary, accuracy increased from 0/192 to 155/192 (80.73%) in one vocabulary world and to 171/192 (89.06%) in a disjoint second world. All 21,348 recorded old responses remained unchanged in these input-only conditions. A configuration that also trained an always-active shared residual module reached 145/192 (75.52%) and lost 92 previously correct old responses. Separate association experiments stored 10,240 synthetic entity–attribute mappings in 80 MiB of input parameters, obtaining 10,182 canonical and 9,963 withheld-wording correct responses. A subsequent isolated 512-association addition preserved all recorded prior skill responses. These results support limited vocabulary transfer and retention under architectural isolation. They do not establish superiority to textual instructions, new general reasoning ability, model-independent memory, or long-project context compression. The release provides saved responses, split checks, historical source snapshots, parameter assets, and a model-free results verifier.

1 Question and scope

The concrete effect studied here is a change in command-conditioned behavior. Before training a new opaque command, the model did not correctly execute the target four-word permutation on any of the 448 recorded skill evaluation queries in each condition. After training only the command's input vector, it executed most queries correctly, including many containing words absent from this skill's training vocabulary. Table 1 gives the complete split-level result; Section 4 shows both successful and failed responses.

The vector is a learned parameter supplied to the model through its input embedding layer. This is not a vector-search database: there is no nearest-neighbor retrieval of answer text at inference. Nor is it a universal skill file. Its coordinates are meaningful relative to the particular frozen reader and tokenizer interface used to learn it. We use *skill binding* for the observed association between a command and a reusable transformation, without claiming the vector creates that transformation from scratch.

The reader already had supervised experience with simpler word-reordering tasks. Consequently, success may reflect access to and recombination of existing capabilities. The contribution of this report is a narrowly scoped empirical record: paired before/after behavior, vocabulary-disjoint evaluation, an interference comparison, association scaling under explicit parameter growth, and deployment fidelity checks. The experiments were development studies with one training seed and a small number of vocabulary worlds; the report is not a preregistered confirmatory trial.

2 Reader and trainable input interface

The base is SmolLM2-1.7B-Instruct [1], pinned to revision

`31b70e2e869a7173562077fd711b654946d38674`.

Its original input vocabulary has 49,152 entries and embedding width 2,048. The reader includes a previously trained, full-layer rank-6 LoRA maintenance adapter, with alpha 12 and dropout zero. The base and maintenance adapter are frozen during the experiments reported in the main comparisons. "Frozen" therefore describes the experimental stage, not an untouched pretrained model.

The maintenance lineage rehearsed existing three-word left rotation and reversal skills alongside general and factual examples. Its final maintenance stage used 432 examples, including 128 original skill teachers. This prior exposure is material: the present four-word transfer task tests new command binding and extension within a familiar class of operations, rather than the emergence of word manipulation without relevant supervision.

Let E_0 be the frozen original embedding table and let B_j contain separately stored learned rows. The effective input embedding is

$$E(t) = \begin{cases} E_0[t], & t < V_0, \\ B_j[t - o_j], & o_j \le t < o_j + |B_j|, \end{cases} \qquad V_0 = 49{,}152,$$

where the added token IDs and offsets o_j are fixed by a lexical inventory. The language-model output head remains the original one. Added tokens act as inputs, not new output vocabulary. Ordinary queries that do not contain the new token do not use its row. With all their existing parameters frozen, this supplies a direct architectural reason for old responses to remain stable.

Synthetic entity identifiers receive separate input rows. Each FP32 row occupies $2{,}048 \times 4 = 8{,}192$ bytes. A command receives one analogous row; thus the skill-specific increment is 8 KiB, excluding the base, maintenance adapter, previously accumulated memory, tokenizer inventory, and runtime overhead. A saved skill checkpoint includes those inherited input banks and is much larger than 8 KiB.

3 Skill experiment

3.1 Task, splits, and conditions

The opaque command `qezorvank` takes four distinct words and requests the order (w_3, w_1, w_4, w_2). Each vocabulary world contains 512 training queries, 64 development queries, 192 withheld combinations from the 64 training words, and 192 transfer queries using a disjoint set of 16 words. All

960 input questions within a world are distinct. Each training word occurs eight times in each of the four input positions. The complete 80-word vocabularies of worlds A and B are disjoint.

"Unseen words" here means excluded from this command's teaching vocabulary. It does not mean unknown to the pretrained tokenizer or absent from pretraining. Transfer words can share subword units with training words. The same permutation and query template are used across splits; this is vocabulary transfer, not a test of novel task instructions or arbitrary composition of skills.

The three conditions start from the same pre-skill reader and 10,240-association checkpoint. Input A and Input B train only the new command row in their respective worlds. Shared A additionally trains rank-2 residuals on the query and value projections in attention layer 23, with alpha 4. These residuals are always active. Together with the command row, this condition has 18,432 trainable FP32 parameters (72 KiB). Input B does not inherit Input A's learned command parameters.

3.2 Optimization and evaluation

Each condition uses 20 teaching epochs followed by 10 additional supervised epochs, historically called a "sleep" phase. The latter is further optimization on the same teacher set, not evidence for biological sleep or a separate consolidation mechanism. With batch size eight, each run performs 1,920 updates and 15,360 example presentations. AdamW with zero weight decay is reset between phases; the schedule has a five-percent warmup followed by cosine decay, with gradient clipping at one. Input-row peak learning rates are 0.003 and 0.001 in the two phases. Shared-residual rates are 0.0003 and 0.0001. No old examples are replayed in these skill runs. Final endpoints are the scheduled final checkpoints, not a best-checkpoint selection within each run.

Historical evaluation uses greedy generation and saved-checkpoint reloads. Skill batches group equal-length prompts without padding, with a 20-token generation limit. The historical decoder strips outer whitespace and lowercases text before storing responses. Our "stored response equality" refers to those strings, not to unprocessed token streams. Scoring further collapses whitespace and strips leading and trailing characters in the set consisting of period, exclamation mark, and space. Exact spelling, membership, multiplicity, and word order otherwise matter. The release verifier reproduces this scorer and all reported primary counts.

The splits were designed for developmental evaluation. Earlier skill attempts, including a narrower version with 132/192 transfer successes, preceded these runs. Therefore the current results must not be presented as an untouched benchmark used only once across the entire research process. No claim of independent replication follows from separate verification code: that code checks archived records, not a fresh neural forward/backward pass.

3.3 Results

Input-only training is associated with a substantial improvement over the untrained command on withheld combinations and command-untrained vocabulary. Because the transfer words are disjoint from the teaching vocabulary, the result cannot be explained solely by replaying the exact teaching answers. It remains consistent with binding a new input vector to a transformation already partly available in the reader.

The Shared A configuration improves the small development split but performs worse on transfer and changes old behavior despite unchanged old input banks. This demonstrates that protecting stored memory rows alone does not protect responses when the reader's shared computation is modified.

Condition	Dev /64	Combinations /192	Transfer /192	Old changed /21,348	Old lost correct
Before: each condition	0	0	0	–	–
Input A	59	190	155 (80.73%)	0	0
Shared A	64	187	145 (75.52%)	1,788	92
Input B	64	191	171 (89.06%)	0	0

Table 1: Complete skill endpoints. "Old changed" counts unequal stored response strings; "Old lost" counts previously correct responses becoming incorrect. The shared condition changes both the trainable architecture and parameter-group learning rates. It is a configuration comparison, not an isolated causal estimate of shared adaptation.

It does not show that all shared adaptation is inferior: the conditions differ in parameterization and learning rates, and there is no hyperparameter-matched family-wide study.

Input A exceeds its historical 80-percent transfer gate by only one correct item (155 rather than the minimum 154 of 192). The 192 queries reuse 16 words and are not independent vocabulary samples. Accordingly, we report counts rather than an item-independent significance test or a broad statistical guarantee. World B gives a second, disjoint lexical test, but uses the same training seed and is not a multi-seed robustness study.

4 What the model actually replied

For each condition we select the first successful and first failed transfer query in the original evaluation order. This fixed rule avoids selecting only attractive successes. The full response records are supplied with the release. In the following examples, *target* is the expected answer and *before/after* are stored responses.

Input A: success
`qezorvank: chisel rye silver barley`
Target: `silver chisel barley rye`
Before: `rye silver barley chisel`
After: `silver chisel barley rye`

Input A: failure
`qezorvank: silver oats pliers wheat`
Target: `pliers silver wheat oats`
Before: `oats wheat pliers`
After: `oats silver pliers wheat`

Shared A: success
`qezorvank: chisel rye silver barley`
Target: `silver chisel barley rye`
Before: `rye silver barley chisel`
After: `silver chisel barley rye`

Shared A: failure
`qezorvank: silver nickel wrench coral`
Target: `wrench silver coral nickel`
Before: `nickel coral silver`
After: `wrench silver coral`

Input B: success
`qezorvank: mongoose piston turbine thyme`
Target: `turbine mongoose thyme piston`
Before: `piston turbine thyme mongoose`
After: `turbine mongoose thyme piston`

Input B: failure
`qezorvank: anchor parsley thyme basil`
Target: `thyme anchor basil parsley`
Before: `parsley thyme basil`
After: `thymus parsley basil anchor`

These failures include wrong order, a missing word, and a non-input substitution. They are not all formatting errors. The successful examples illustrate a usable narrow transformation; the failed examples constrain any claim of reliable general execution.

5 Association storage and subsequent learning

5.1 10,240 associations

A separate sequence learns synthetic entity–attribute mappings in 20 isolated banks of 512 rows. The full FP32 input-bank payload is 20,971,520 parameters, or 80 MiB. At the scale endpoint, canonical questions are correct for 10,182/10,240 mappings (99.43%), and the designated withheld wording is correct for 9,963/10,240 (97.29%).

A more conservative count compares each cohort with its actual pre-learning responses. There are 9,089 mappings for which both canonical and withheld-wording responses were wrong at introduction and both are correct at the final endpoint. This avoids treating every final correct answer as a newly acquired association. The release contains all 20 introduction baselines, including the first two cohorts inherited by the scale continuation. Across the recorded 2,916 old questions preceding the continuation, stored responses remain unchanged.

The experiment allocates one new row per entity. It is therefore a demonstration of separately parameterized association storage, not a fixed-capacity compression result. The identifiers, lexical inventory, frozen reader, and maintenance adapter are required in addition to the numerical rows. A historical compact textual representation uses 39,751 tokens, about 4.85 times the reader's 8,192-token context, but that arithmetic is not a fair retrieval comparison or a proof of compression. Neither a text-retrieval baseline nor an optimized textual representation was evaluated. A separate earlier prospective wording test reached 483/512 (94.34%), below its 95-percent gate; high performance on the designated wording does not establish arbitrary paraphrase robustness.

5.2 Adding 512 associations after the skill

Starting from Input A, a later run adds 512 new rows (4 MiB FP32) while freezing all prior banks, the skill row, and the reader. It obtains 508/512 canonical and 496/512 withheld-wording correct responses, with 447 conservative new acquisitions by the same both-wrong-to-both-right rule. All 21,348 prior responses and all 448 skill responses remain identical to their recorded pre-addition strings, including prior errors.

This is successful retention under explicit isolation. Since queries for older entities and the old skill do not use the newly trained rows, it is an expected benefit of the interface, not evidence that shared

parameters learned without interference. It also does not demonstrate that factual associations have become procedural skills. The association and permutation tasks provide two distinct uses of the same input-parameter mechanism.

6 Deployment and resource accounting

A deployment-only experiment stores the 10,240-association banks in FP16, reducing their tensor payload from 80 to 40 MiB. Reloading the deployment artifact preserves all 21,348 recorded responses. The serialized FP16 input file is 41,949,170 bytes; 41,943,040 of these bytes are tensor payload. The frozen model, adapter, lexical metadata, and original FP32 master checkpoint are additional costs.

The original forward path already casts the bank vectors to the reader's FP16 embedding dtype. The successful deployment export preserves those effective values using the matching DirectML conversion. An earlier CPU-conversion attempt did not satisfy the equivalence check. The result concerns storing already deployed values faithfully; it is not evidence that arbitrary quantization preserves knowledge. Training resumption from FP16-only assets was not validated. Peak GPU allocator data were unavailable, not zero; no peak-memory or throughput advantage is claimed from these records.

Run	Updates	Presentations	Target tokens	Train seconds
Scale continuation (18 banks)	69,120	552,960	1,868,340	24,058.44
Input A	1,920	15,360	103,920	704.85
Shared A	1,920	15,360	103,920	725.47
Input B	1,920	15,360	105,600	712.87
Later 512	3,840	30,720	103,680	1,384.76

Table 2: Historical producer-recorded optimization costs. Target tokens count supervised positions. Scale costs exclude the inherited first two banks. Times exclude some preparation, validation, and prior development, and are not end-to-end benchmark timings.

The deployment run performs zero updates and records 1,804.72 seconds including loading, casting, and generation. These experiments used the historical Windows/DirectML execution path. Timing is included as provenance, not as a portable hardware-efficiency claim. The maintenance lineage and earlier failed experiments impose further costs not included in Table 2.

7 Related work and interpretation

Continuous learned inputs for frozen language models have established precedents. Prefix-Tuning learns continuous prefixes to condition a frozen model [2]; Prompt Tuning studies learned soft prompts and the effect of model scale [3]. Our single command row is closely related to this family. We do not claim to introduce parameter-efficient prompting or to establish a new general memory architecture.

Gist tokens compress instructions into learned representations produced by a trained model [4]. AutoCompressors produce soft summary vectors for longer text [5], and the In-context Autoencoder learns compact memory representations of context [6]. These works make context or instruction compression a substantive existing research area. In contrast, this report optimizes model-specific

rows for synthetic associations or a particular command. It contains no comparison showing superior compression, latency, accuracy, or amortized cost relative to those methods.

An external parameter file can be small relative to its reader while still being expensive relative to a short textual instruction. For this permutation, a concise instruction such as "return the third, first, fourth, then second word" is an essential missing baseline. Therefore an 8 KiB vector and a short command do not by themselves establish an advantage over document-based skills. The most defensible current result is narrower: the adapted reader can be conditioned through a learned input row to execute this transformation on many command-untrained words, with architectural isolation preserving recorded prior behavior.

8 Limitations and next tests

The largest limitations are prior related-task exposure in the reader; one training seed; two related, small vocabulary worlds; short synthetic tasks; and repeated developmental access to evaluation panels. Greedy string scoring does not assess open-ended semantic usefulness, calibration, long-horizon planning, or robustness to adversarial inputs. Retention of a finite old panel is not retention of all language ability. Saved source hashes and numerical checks increase traceability but do not replace an independent implementation and fresh rerun.

No same-model textual-skill, retrieval, or full-context baseline was run. No cross-backbone transfer was tested, either with unchanged vectors or with newly trained model-specific vectors. No long factual passage or agent project history was converted into a useful continuing-work memory. There is also no demonstrated causal contribution of the extra supervised "sleep" phase. These missing comparisons prevent claims that the method solves context compression or surpasses conventional agent skills.

A useful next study would preregister unseen tasks and vocabularies, compare the same reader under concise instructions, retrieved documents, and learned vectors, and record both training cost and inference context/runtime cost. It should vary seeds and tasks, distinguish untouched-base from adapted-reader results, and test a fixed update policy on genuinely held-out materials. Such work is proposed here, not reported as completed.

9 Evidence availability and reproducibility boundary

The accompanying release contains six compressed exports with saved question, expected-answer, and response records; training/evaluation split archives; all 20 scale introduction baselines; historical independent-audit records; source hashes; and 76 archived Python files with their locally resolvable dependencies. The standard-library verifier recomputes primary scores, response pairings, conservative acquisition counts, vocabulary separation, exposure balance, and source integrity without loading a model. This is *evidence-level reproduction.*

Separately packaged parameter assets include the input banks, lexical inventories, the shared residual when applicable, and the frozen maintenance adapter. The original base-model weights are not redistributed; the pinned upstream revision is specified instead. Exported inference configurations relocate the maintenance-adapter path; their new hashes are recorded. Historical source snapshots retain their original assumptions and are not advertised as a turnkey clean-machine training pipeline. No new neural run was performed for preparing this release. Historical audits are preserved evidence,

not freshly rerun certificates.

Machine-local paths in JSON exports are replaced with neutral placeholders. Response strings are preserved; exports are reserialized and therefore have new hashes, recorded separately from the original-file hashes. Complete experimental success criteria and failed conditions are not rewritten for publication. This report selects a bounded completed study from a broader development history and does not claim that the broader project's original objectives were achieved.

10 Conclusion

In this adapted 1.7B reader, an 8 KiB trainable command input supports a clear but incomplete improvement in four-word permutation on vocabulary excluded from command teaching: 155/192 and 171/192 correct responses from zero baselines. Isolated parameter additions preserve the recorded old responses, whereas the tested always-active shared update changes old behavior and loses correct answers. The association and deployment results establish additional, bounded engineering capabilities. The evidence supports further controlled evaluation of model-specific learned input memories, while leaving their advantage over textual skills and their usefulness for long-running agents unresolved.

AI assistance and responsibility

OpenAI Codex assisted with software, evidence inspection, numerical verification, manuscript preparation, and release packaging. ByteDance Seed2.1 (Doubao) contributed to earlier AI-assisted project work, as reported by the author; the exact serving variant and a complete task-level attribution log are not available. These systems are acknowledged tools, not authors. Shenghe Li is the human author and is responsible for the manuscript, interpretations, and release decisions. AI-assisted checking is not represented as independent human peer review.

References

[1] HuggingFaceTB. *SmolLM2-1.7B-Instruct*. Model card and pinned model repository. https://huggingface.co/HuggingFaceTB/SmolLM2-1.7B-Instruct.

[2] Xiang Lisa Li and Percy Liang. 2021. Prefix-Tuning: Optimizing Continuous Prompts for Generation. *ACL-IJCNLP*. https://arxiv.org/abs/2101.00190.

[3] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning. *EMNLP*. https://arxiv.org/abs/2104.08691.

[4] Jesse Mu, Xiang Lisa Li, and Noah D. Goodman. 2023. Learning to Compress Prompts with Gist Tokens. *NeurIPS*. https://arxiv.org/abs/2304.08467.

[5] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. 2023. Adapting Language Models to Compress Contexts. *EMNLP*. https://arxiv.org/abs/2305.14788.

[6] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. 2024. In-context Autoencoder for Context Compression in a Large Language Model. *ICLR*. https://arxiv.org/abs/2307.06945.